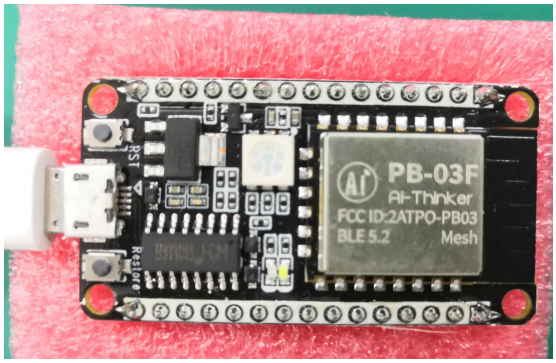
PB-3F board Development guidance

1. Hardware introduce

PB-03F-Kit is the development board of PB-03F



PB-03 module introduction

PB-03 is a Bluetooth module developed by Shenzhen Ai-Thinker(Anxinke) Technology Co., LTD. The PHY6252, a highly integrated low-power Bluetooth system-level chip (SoC), is designed for the Internet of Things (IoT), mobile devices, wearable electronic devices, smart home and other applications.

Support BLE5.2, inside the chip 64 KB SRAM，256KB flash，96 KB ROM.

1.1. Characteristic

- Support BLE5.2,rate support:125Kbps,500Kbps,1Mbps,2Mbps
- Own 64 KB SRAM，256KB flash,96 KB ROM，256bit efuse
- Support UART/GPIO/ADC/PWM/I2C/SPI/PDM/DMA interface
- Adopt SMD-22 package
- Support multiple sleep modes,deep sleep current is less than 1uA
- Support for serial local upgrade and remote Firmware upgrade(FOTA)
- Universal AT instructions can be used easy and quickly
- Support for secondary development,with an integrated Windows development environment

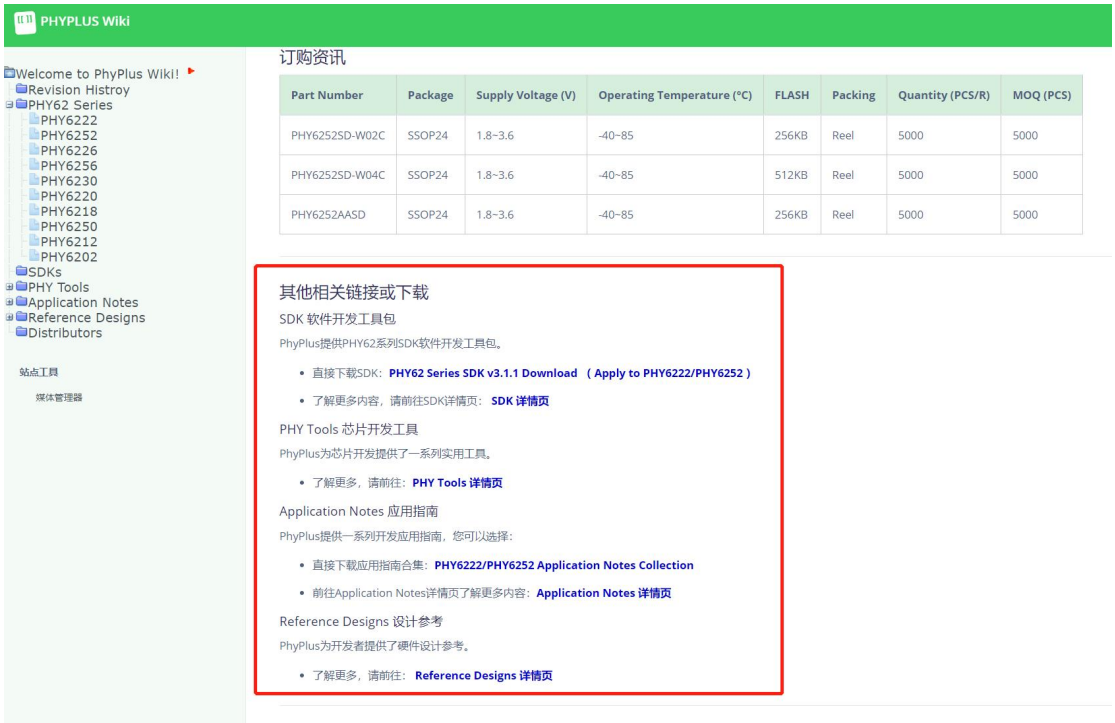
Ai-Thinker official website:

https://docs.ai-thinker.com/en/blue_tooth_pb

PHYplus website

https://www.phyplusinc.com

http://wiki.phyplusinc.com/doku.php?id=menu:phy62_series:phy6252



Part Number	Package	Supply Voltage (V)	Operating Temperature (°C)	FLASH	Packing	Quantity (PCS/R)	MOQ (PCS)
PHY6252SD-W02C	SSOP24	1.8~3.6	-40~85	256KB	Reel	5000	5000
PHY6252SD-W04C	SSOP24	1.8~3.6	-40~85	512KB	Reel	5000	5000
PHY6252AASD	SSOP24	1.8~3.6	-40~85	256KB	Reel	5000	5000

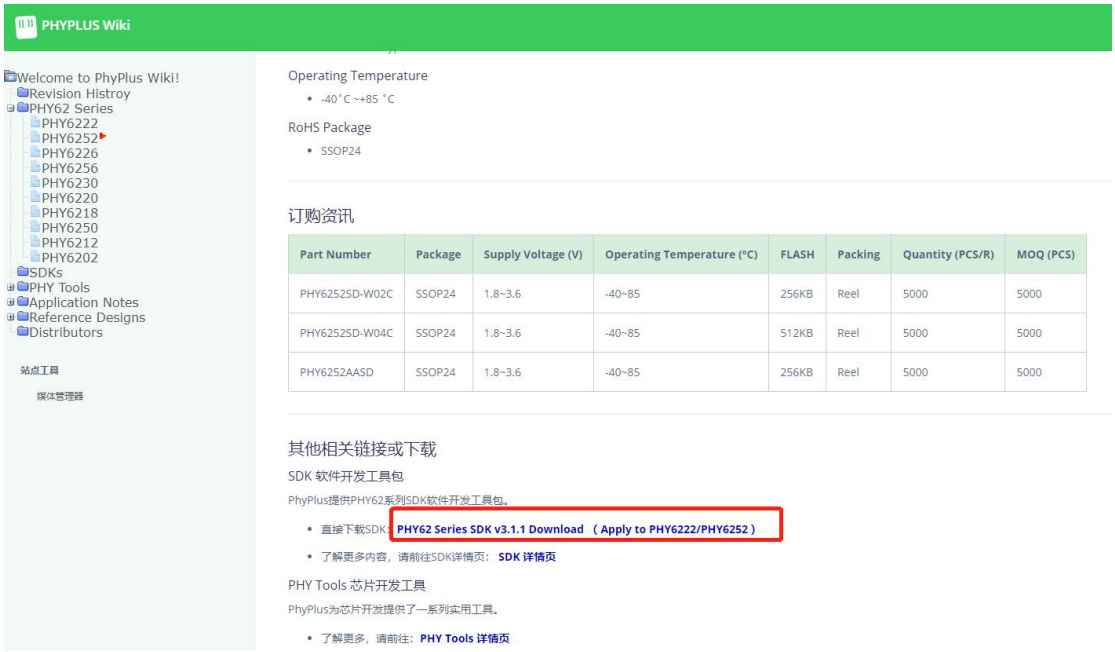
2.Build development environment

The development environment mainly uses the Keil 5 integrated development software.

2.1 Keil5 software install

2.2 SDK download

Download SDK on website and then unzip

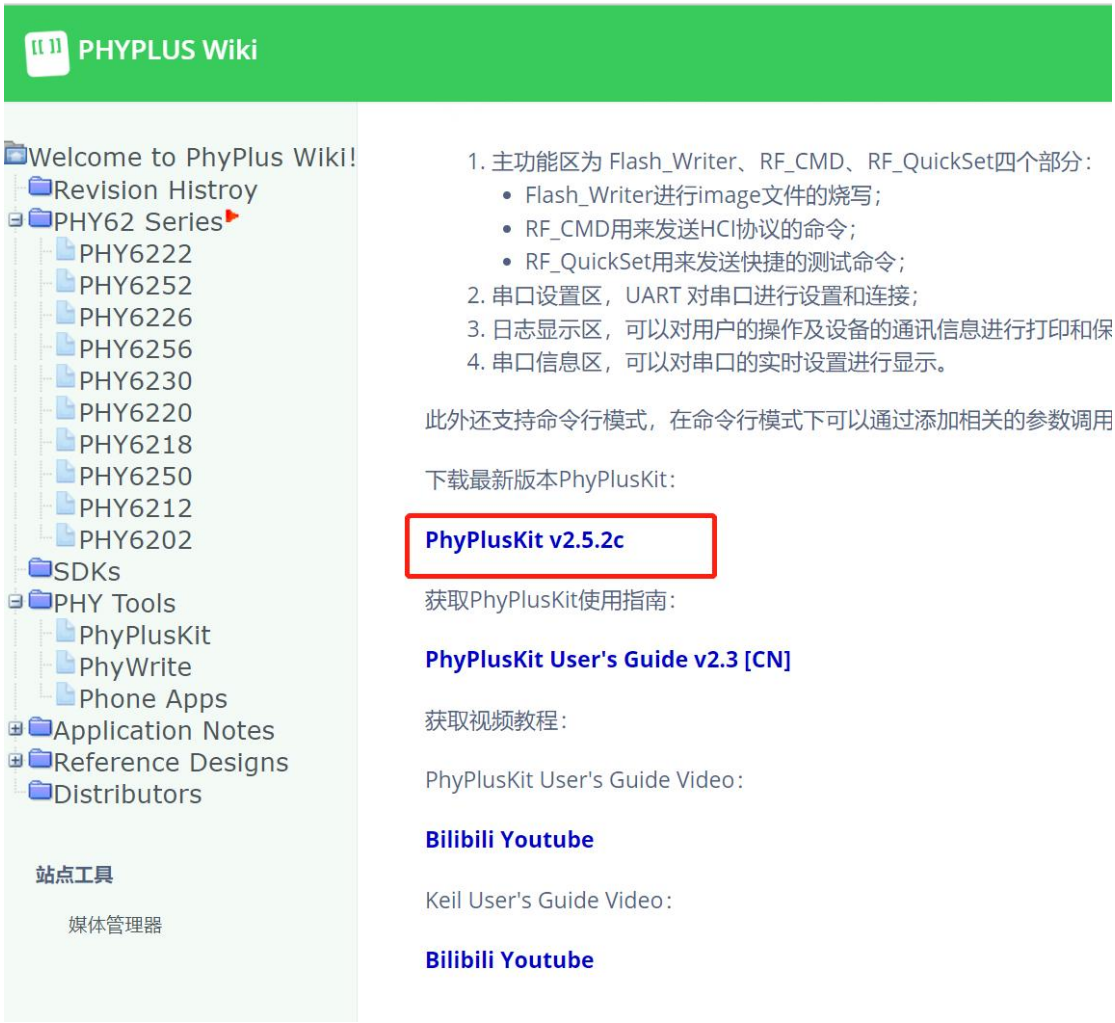


SDK directory structure

```
PHY62XXSDK
├─components                  ;SDK组件，包括BLE API，GATT Profiles，芯片硬件驱动和其他软件组件
├─example                     ;例程
│  ├─ble_central             ;
│  ├─ble_peripheral          ;
│  │  ├─alternate_iBeacon    ;alternate iBeacon样例
│  │  ├─ancs                 ;苹果消息中心（ANCS）例程
│  │  ├─bleI2C_RawPass       ;I2C透传例程
│  │  ├─bleSmartPeripheral   ;综合Peripheral例程
│  │  ├─bleUart-RawPass      ;UART透传例程
│  │  ├─eddystone            ;eddystone例程
│  │  ├─HIDKeyboard          ;HID例程
│  │  ├─hrs                  ;Heart rate profile演示例程
│  │  ├─iBeacon              ;iBeacon例程
│  │  ├─otaDemo              ;最基本的OTA演示例程
│  │  ├─pwmLight             ;通过BLE命令控制PWM进行LED亮度调整的例程
│  │  ├─RawAdv               ;简单的广播例程，用于无线胎压监测一类的应用
│  │  ├─Sensor_Broadcast     ;
│  │  ├─wrist                ;综合样例，用于运动手环一类的应用
│  │  ├─wrist_aptm           ;基于综合样例，通过AP Timer+OSAL Timer实现实时性较高的时钟
│  │  └─XIPDemo              ;部分实时性要求较低的代码直接运行于内部flash的例程
│  ├─OTA                     ;
│  │  ├─OTA_internal_flash   ;OTA bootloader
│  │  └─OTA_upgrade_2ndboot  ;特殊应用，用于升级OTA bootloader自身
│  └─peripheral              ;
│     ├─adc                  ;ADC驱动应用样例
│     ├─ap_timer             ;AP Timer驱动应用样例
│     ├─fs                   ;文件系统样例
│     ├─gpio                 ;GPIO演示例程
│     ├─kscan                ;4x4按键演示例程
│     ├─lcd_ST7789VW         ;240x240 TFT彩屏演示例程
│     ├─pwm                  ;PWM演示例程
│     ├─qdec                 ;QDEC演示例程
│     ├─spiflash             ;外部SPI演示例程
│     ├─voice                ;语音采集演示例程
│     ├─voice_sbc            ;SBC格式语音采集编码演示例程
│     └─watchdog             ;看门狗演示例程
├─lib                        ;lib文件和.h文件，包括蓝牙协议栈更新，字库
│  └─font                    ;字库的资源文件和升级文件
└─misc                       ;ROM symble table和跳转表
```

2.3. Download program tool

Download PhyPlusKit software



PhyPlusKit User's Guide Video： [Youtube]

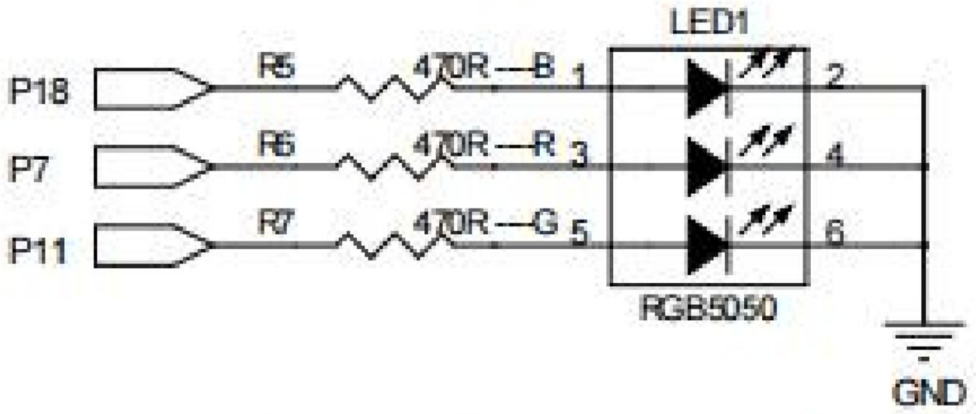
Keil User's Guide Video： [Youtube]

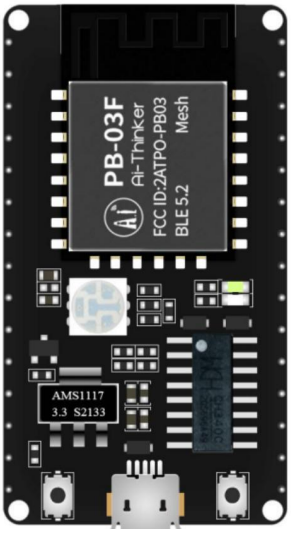
3.Lighting program (Embedded hello world)

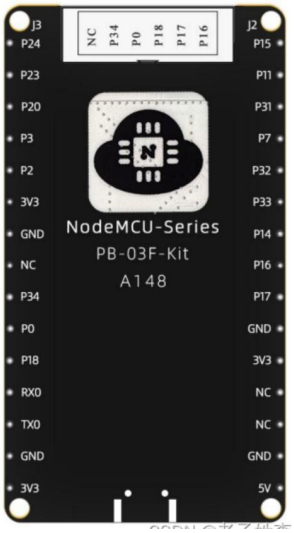
3.1 LED Schematic

The development board contains an RGB light with the schematic shown below.

We'll light up the blue (Blue) LED, so the code needs to control the high level of the P18 (GPIO _ 18) output.







Indicator light or key	LED state and key function
RGB light	Red light (P7)
	Gree light (P11)
	Blue lighe (P18)
White light	Cool light(P34)
Yellow light	Warm light (P0)
Restore key	Restore (P15)
RST key	Rstart

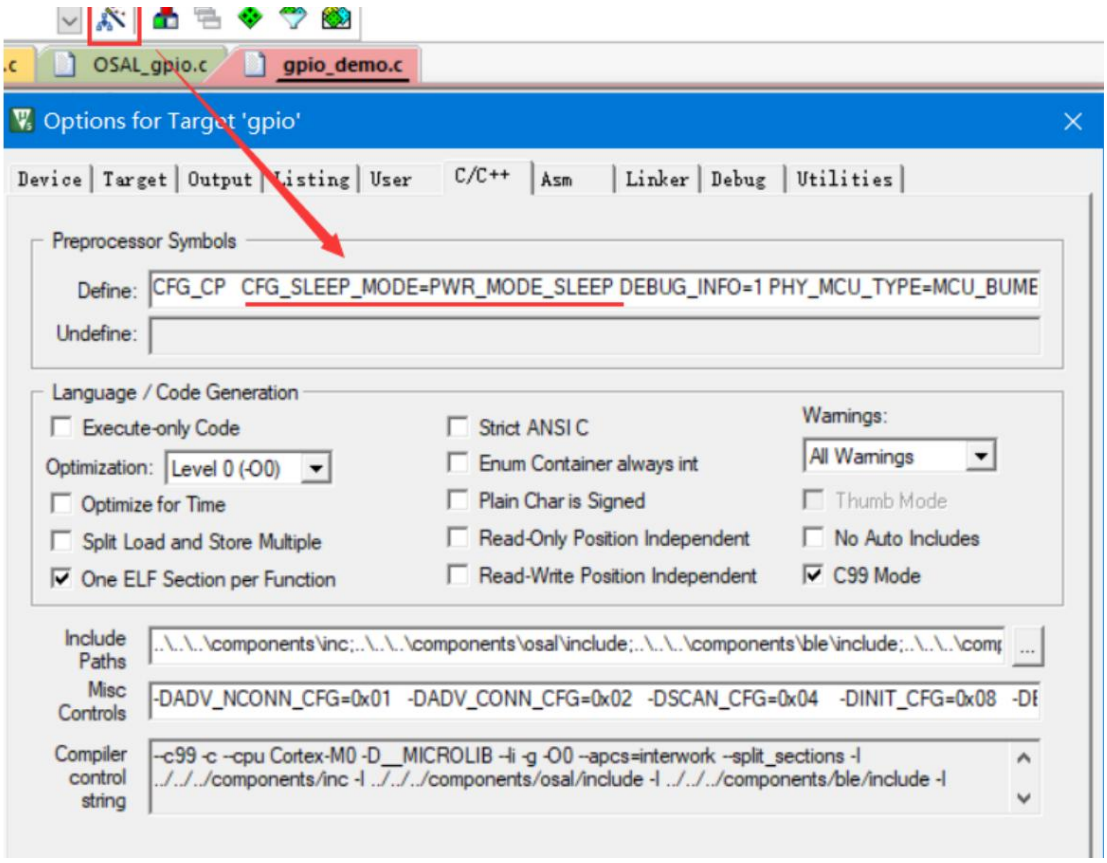
3.2 Code writing

Change the code based on the gpie routine
Open the example code example\peripheral\gpio under SDK，Double click the project file.

Suggest to turn off the low-power mode

Because the chip is dormant, the LED output also turns off.

Click on the magic wand and select the C / C + + option.



Macro declaration ：

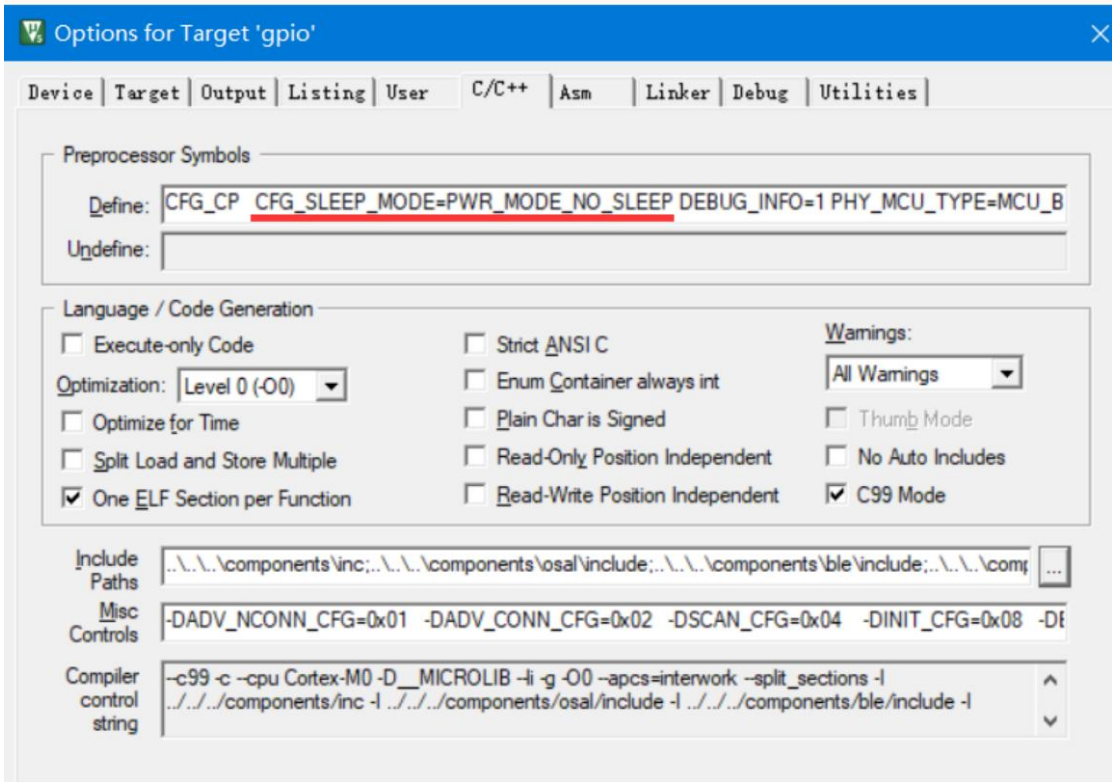
CFG_SLEEP_MODE=PWR_MODE_SLEEP ： Enable low power mode, firmware program execution, it will enter into sleep in the idle process. After sleep, the

debugger cannot be debug and track, and the breakpoint also fails

CFG_SLEEP_MODE=PWR_MOD_NO_SLEEP ： Close the low power mode and the processor is awake during the firmware execution.

Modify: CFG_SLEEP_MODE=PWR_MODE_SLEEPas CFG_SLEEP_MODE=PWR_MOD_NO_SLEEP

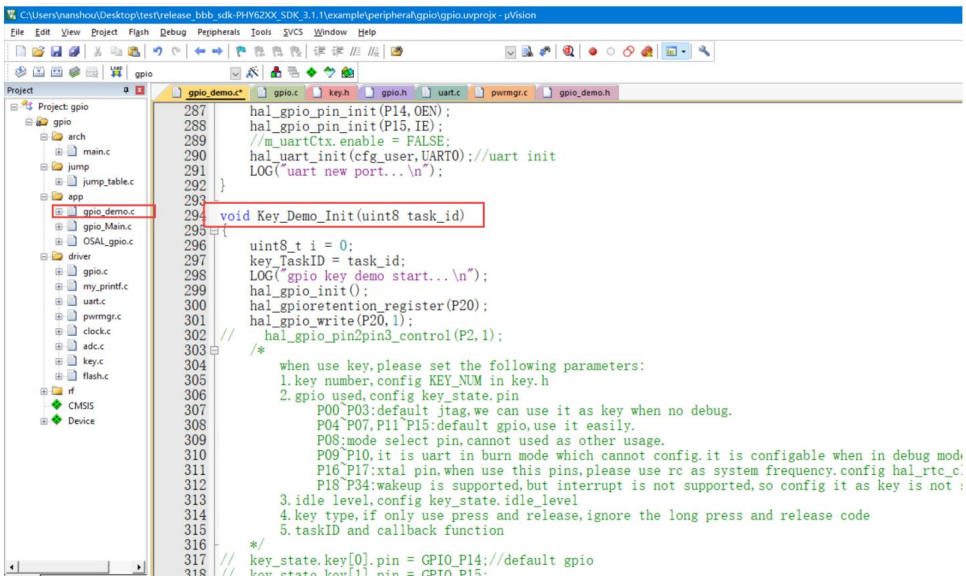
After modify:



Modify gpio_demo.c function

The gpio example demonstrates the key function, and the function written here is too complicated.

We can simplify it to light blue LED (GPIO _ 18 output high level)
Find void Key_Demo_Init(uint8 task_id) to comment it all out.



Replace as below,

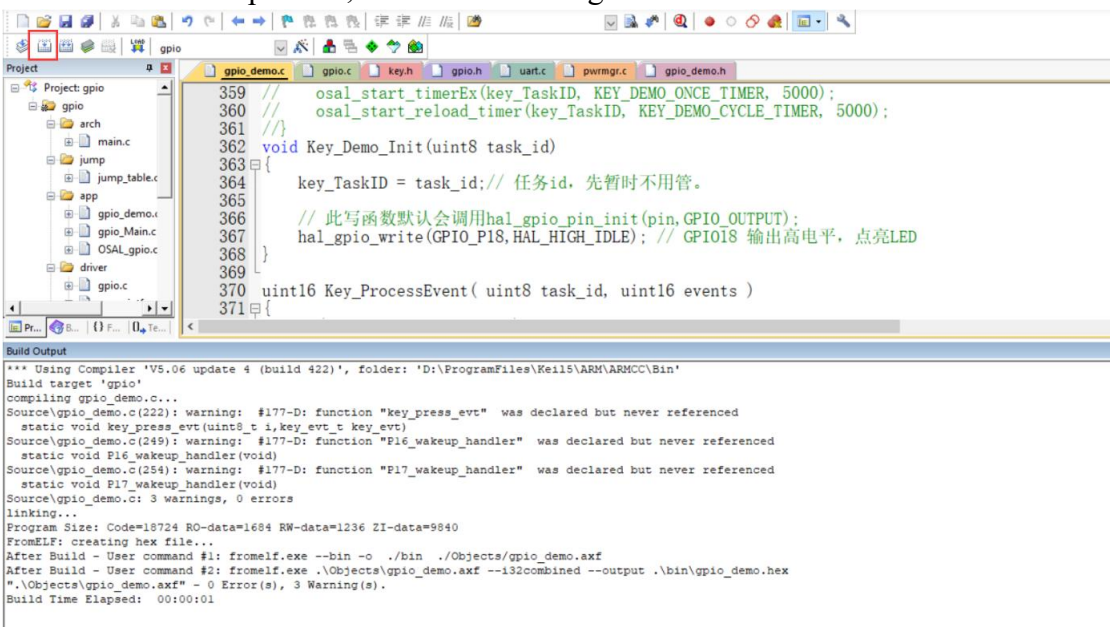
```
//      key_state.key_callbank = key_press_evt;
//      key_init();
//      osal_start_timerEx(key_TaskID, KEY_DEMO_ONCE_TIMER, 5000);
//      osal_start_reload_timer(key_TaskID, KEY_DEMO_CYCLE_TIMER, 5000);
//}
void Key_Demo_Init(uint8 task_id)
{
    key_TaskID = task_id;// 任务id，先暂时不用管。

    // 此写函数默认会调用hal_gpio_pin_init(pin,GPIO_OUTPUT);
    hal_gpio_write(GPIO_P18,HAL_HIGH_IDLE); // GPIO18 输出高电平，点亮LED
}
```

Refer code after modify,

```
//void Key_Demo_Init(uint8 task_id)
//{
//    uint8_t i = 0;
//    key_TaskID = task_id;
//    LOG("gpio key demo start...\n");
//    hal_gpio_init();
//    hal_gpioretention_register(P20);
//    hal_gpio_write(P20,1);
    hal_gpio_pin2pin3_control(P2,1);
//    /*
//        when use key,please set the following parameters:
//        1.key number,config KEY_NUM in key.h
//        2.gpio used,config key_state.pin
//            P00~P03:default jtag,we can use it as key when no debug.
//            P04~P07,P11~P15:default gpio,use it easily.
//            P08:mode select pin,cannot used as other usage.
//            P09~P10,it is uart in burn mode which cannot config.it is configable when in debug mode.
//            P16~P17:xtal pin,when use this pins,please use rc as system frequency.config hal_rtc_clock_config(CLK
//            P18~P34:wakeup is supported,but interrupt is not supported,so config it as key is not suggested.
//        3.idle level,config key_state.idle_level
//        4.key type,if only use press and release,ignore the long press and release code
//        5.taskID and callback function
//    */
  key_state.key[0].pin = GPIO_P14;//default gpio
```

```
48 //        }
49 //        else if(key_state.key[i].pin == GPIO_P17)
50 //        {
51 //            hal_pwrmgr_register(MOD_USR3,NULL,P17_wakeup_handler);
52 //            hal_gpio_cfg_analog_io(key_state.key[i].pin,Bit_DISABLE);
53 //            LOG("P17 is used\n");
54 //        }
55 //        else if((key_state.key[i].pin == GPIO_P09) || (key_state.key[i].pin == GPIO_P10))
56 //        {
57 //            uart_port_reconfig();
58 //        }
59 //    }
60
61   key_state.key[0].idle_level = HAL_LOW_IDLE;
62   key_state.key[1].idle_level = HAL_HIGH_IDLE;
63 //    key_state.task_id = key_TaskID;
64 //    key_state.key_callbank = key_press_evt;
65 //    key_init();
66 //    osal_start_timerEx(key_TaskID, KEY_DEMO_ONCE_TIMER, 5000);
67 //    osal_start_reload_timer(key_TaskID, KEY_DEMO_CYCLE_TIMER, 5000);
68 //}
69 void Key_Demo_Init(uint8 task_id)
70 {
71     key_TaskID = task_id;// 任务id，先暂时不用管。
72
73     // 此写函数默认会调用hal_gpio_pin_init(pin,GPIO_OUTPUT);
74     hal_gpio_write(GPIO_P18,HAL_HIGH_IDLE); // GPIO18 输出高电平，点亮LED
75 }
```

Then click on Compilation, as shown in the figure below.



The compilation here has 3 warnings because this c file defines a function with no call. Lighting the let can temporarily ignore this warning.

```
Build Output
*** Using Compiler 'V5.06 update 4 (build 422)', folder: 'D:\ProgramFiles\Keil5\ARM\ARMCC\Bin'
Build target 'gpio'
compiling gpio_demo.c...
Source\gpio_demo.c(222): warning:  #177-D: function "key_press_evt"  was declared but never referenced
  static void key_press_evt(uint8_t i,key_evt_t key_evt)
Source\gpio_demo.c(249): warning:  #177-D: function "P16_wakeup_handler"  was declared but never referenced
  static void P16_wakeup_handler(void)
Source\gpio_demo.c(254): warning:  #177-D: function "P17_wakeup_handler"  was declared but never referenced
  static void P17_wakeup_handler(void)
Source\gpio_demo.c: 3 warnings, 0 errors
linking...
Program Size: Code=18724 RO-data=1684 RW-data=1236 ZI-data=9840
FromELF: creating hex file...
After Build - User command #1: fromelf.exe --bin -o  ./bin  ./Objects/gpio_demo.axf
After Build - User command #2: fromelf.exe .\Objects\gpio_demo.axf --i32combined --output .\bin\gpio_demo.hex
".\Objects\gpio_demo.axf" - 0 Error(s), 3 Warning(s).
Build Time Elapsed:  00:00:01
```

After Build - User command #2: fromelf.exe .\Objects\gpio_demo.axf --i32combined --output .\bin\gpio_demo.hex

Note for this line--output .\bin\gpio_demo.hex, it tell us the output gpio_demo.hex firmare is under the bin directory.

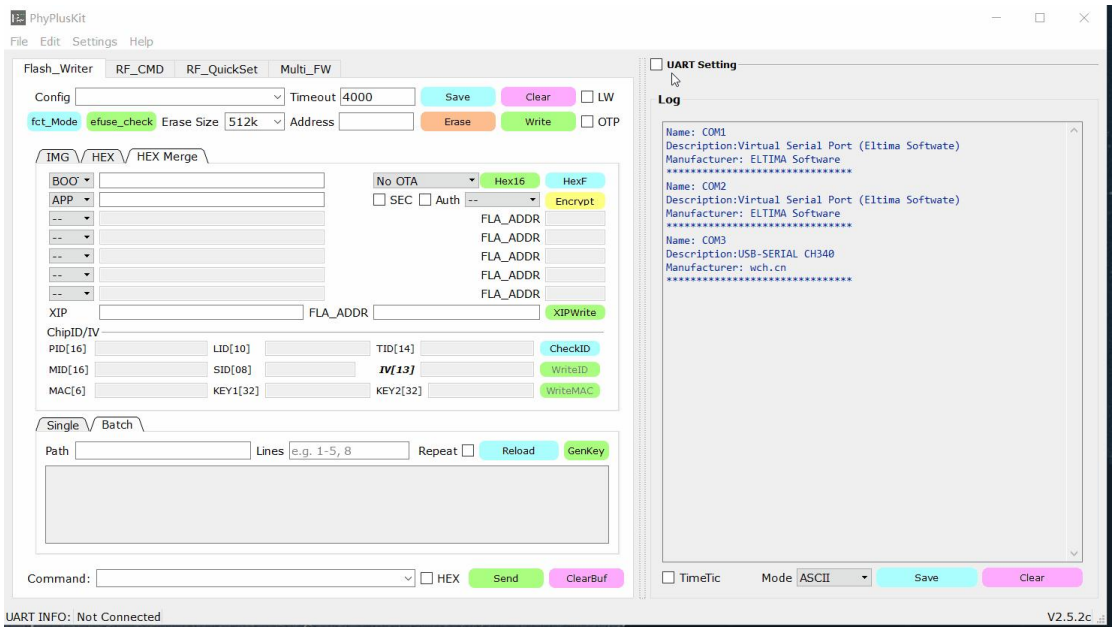
4. Firmware program

4.1 Install UART Driver

The development board driver chip is CH340， so before program you need to install CH340 driver

4.2 Program tool

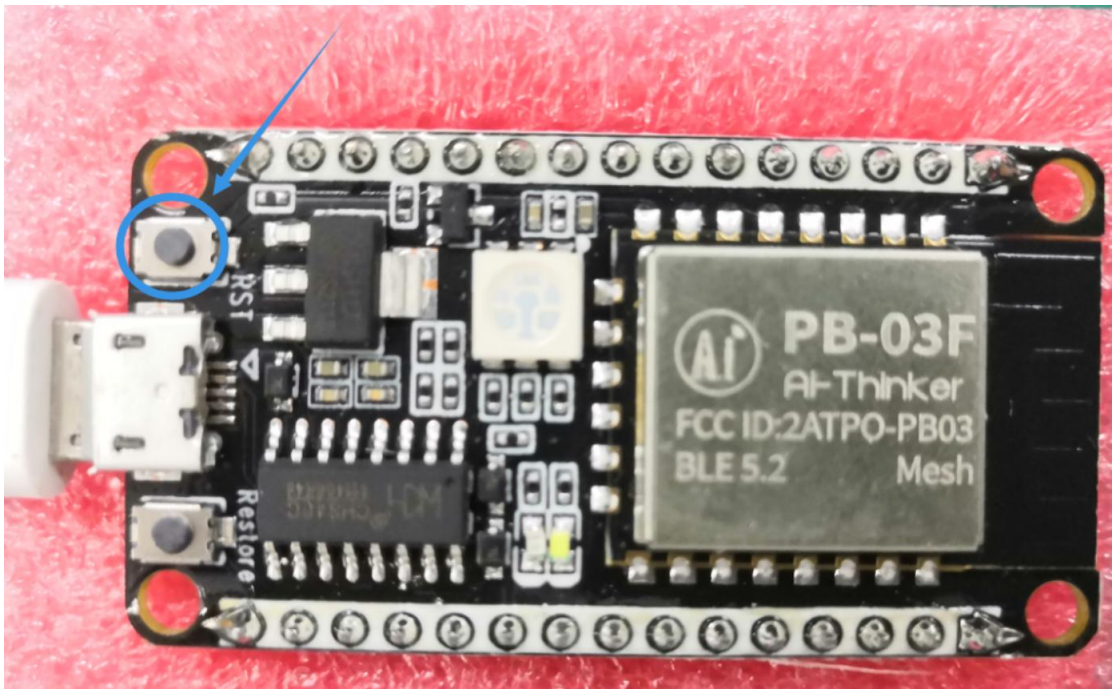
Connect the development，open PhyPlusKit software. Open UART COM3，choose UXTDW.



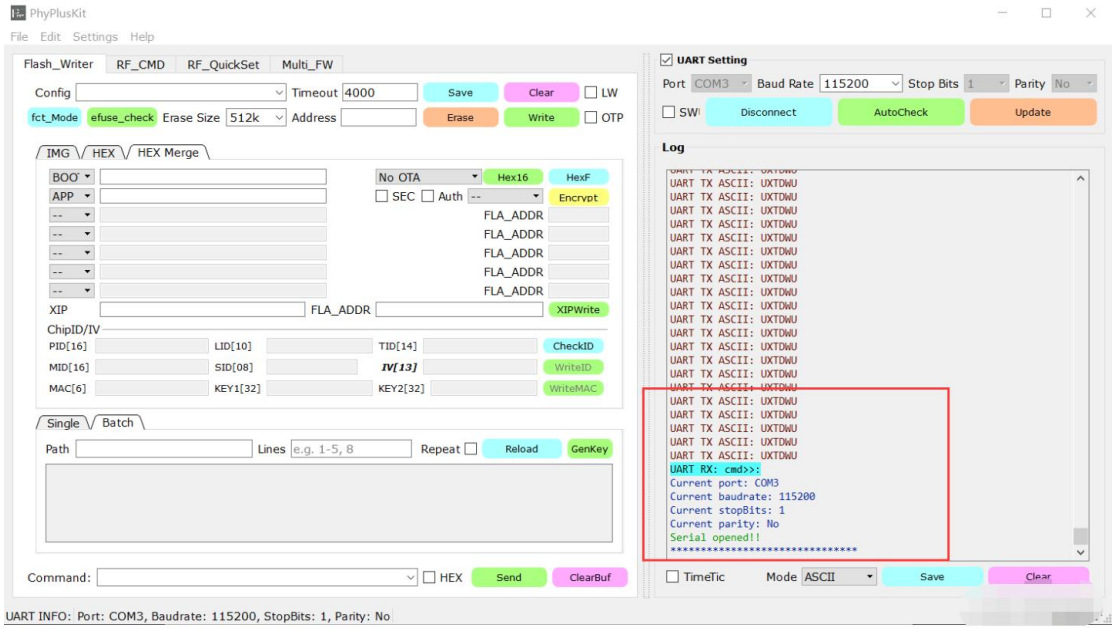
You will see the right side of the output all the time. Shows that the development board has been connected successfully.

```
UART TX ASCII: UXTDWU
UART TX ASCII: UXTDWU
UART TX ASCII: UXTDWU
UART TX ASCII: UXTDWU
UART TX ASCII: UXTDWU
UART TX ASCII: UXTDWU
UART TX ASCII: UXTDWU
UART TX ASCII: UXTDWU
UART TX ASCII: UXTDWU
UART TX ASCII: UXTDWU
UART TX ASCII: UXTDWU
UART TX ASCII: UXTDWU
UART TX ASCII: UXTDWU
UART TX ASCII: UXTDWU
UART TX ASCII: UXTDWU
UART TX ASCII: UXTDWU
UART TX ASCII: UXTDWU
UART TX ASCII: UXTDWU
UART TX ASCII: UXTDWU
UART TX ASCII: UXTDWU
```

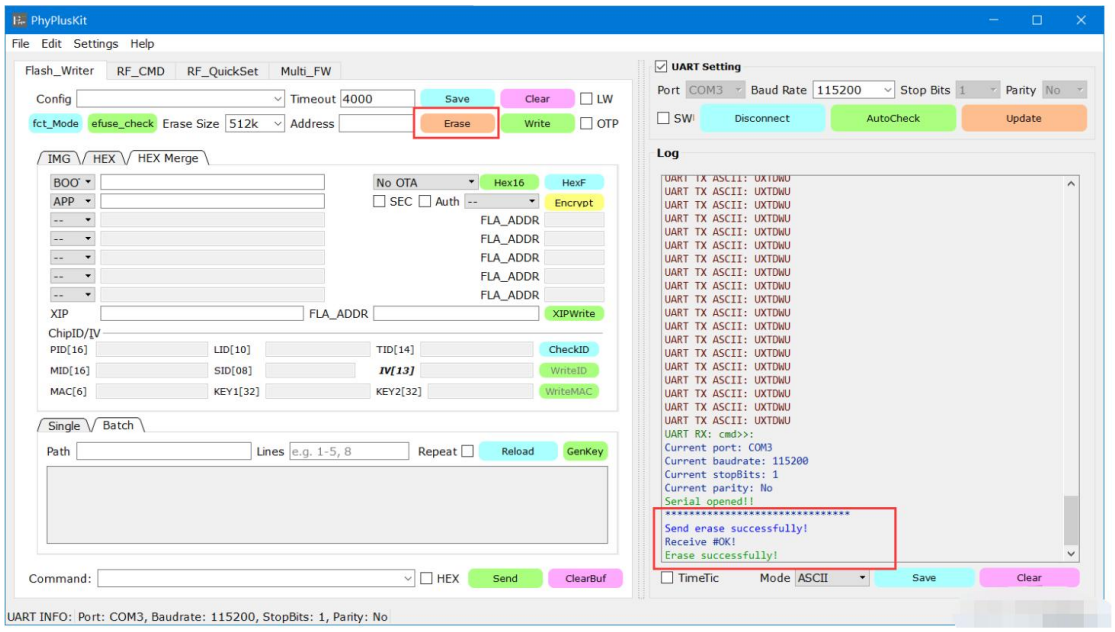
Long press the reset button, about 2s, release the reset button., shown as below image.



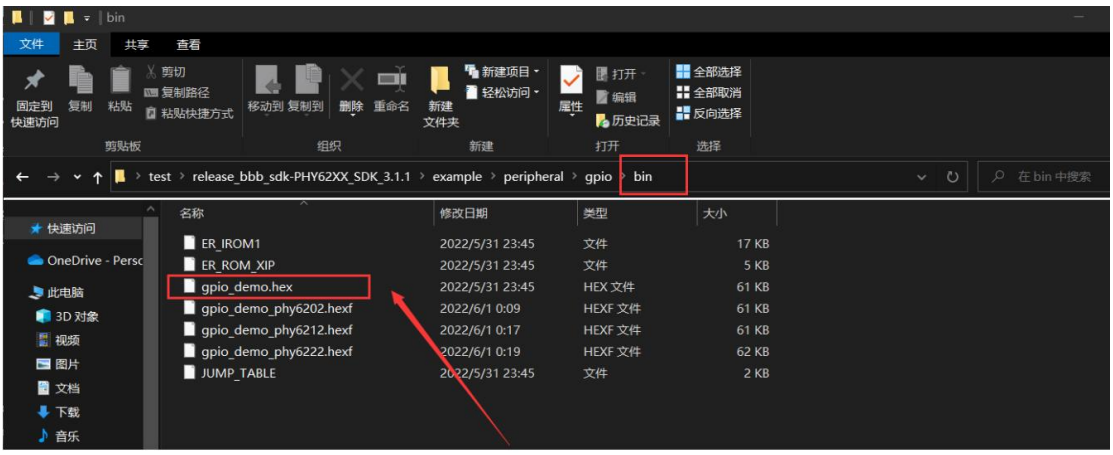
Shows UART RX: cmd>>:，means that you can start download the program. The port rate option automatically changes to 11520, which is a normal phenomenon.



The program must be erased before burning it. Click on: Erase. Erase is as follows.

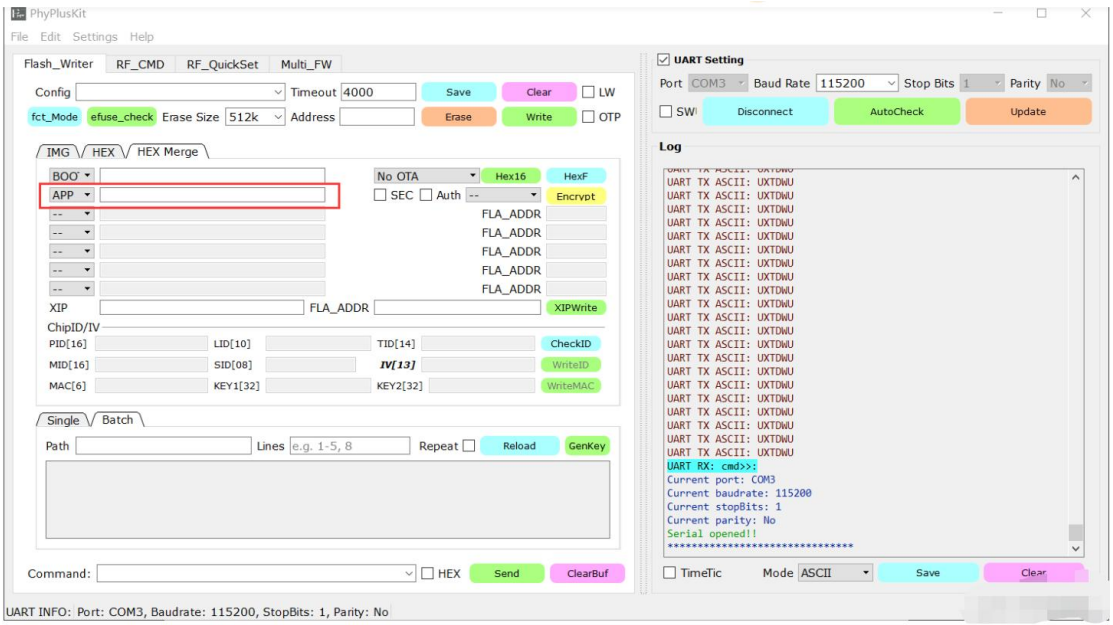


The compile-generated firmware is in the bin directory.

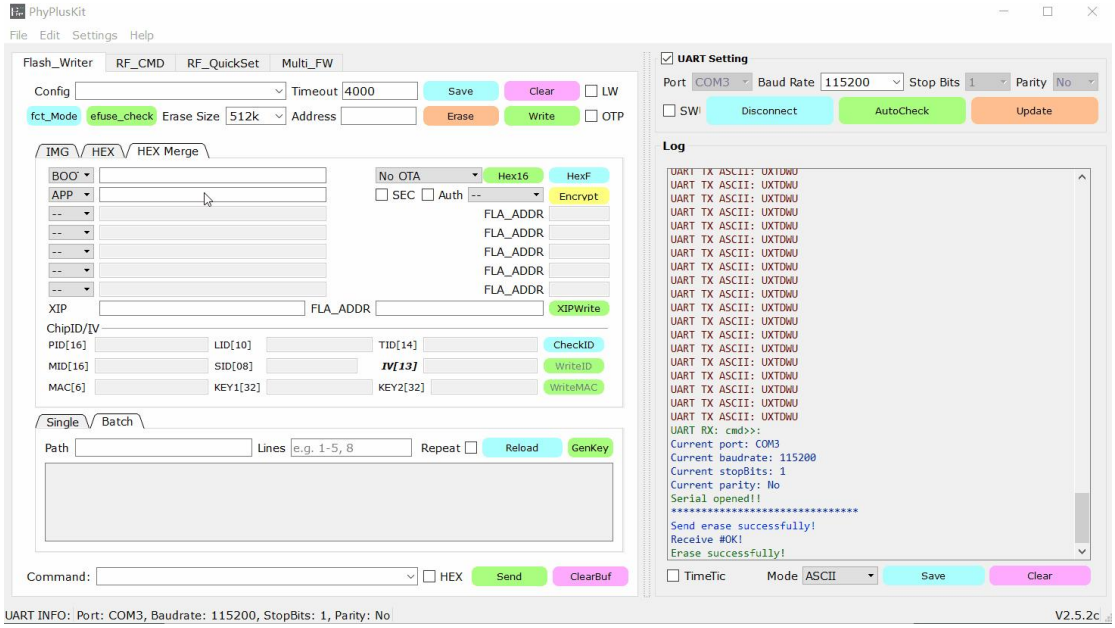


Select the hex firmware

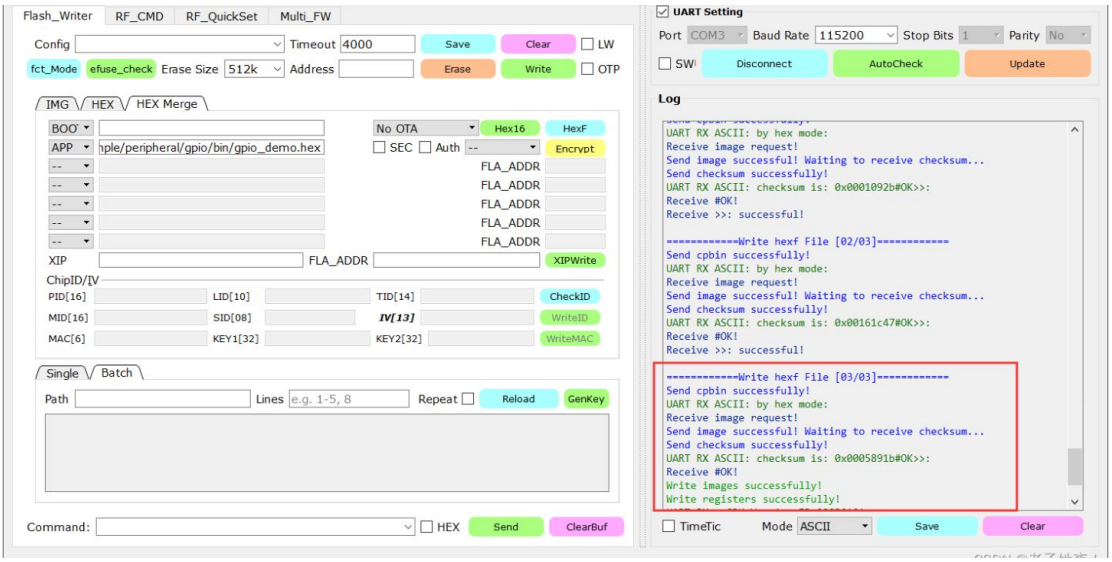
Double-click the input box in the figure below to select the download. hex firmware. The firmware location is shown above.



Click Write， for program burning(flash)



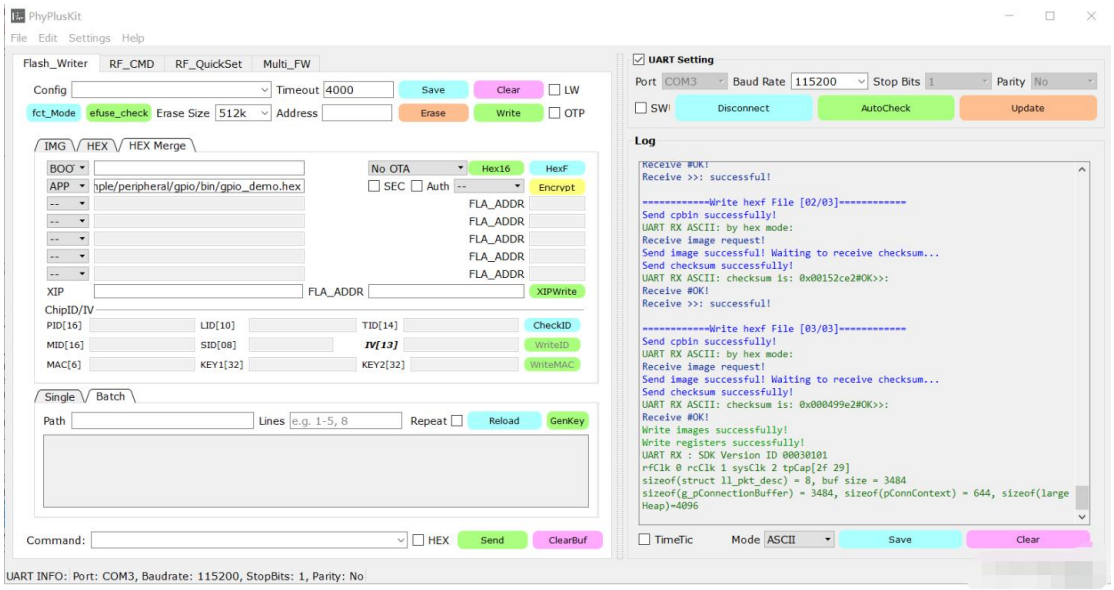
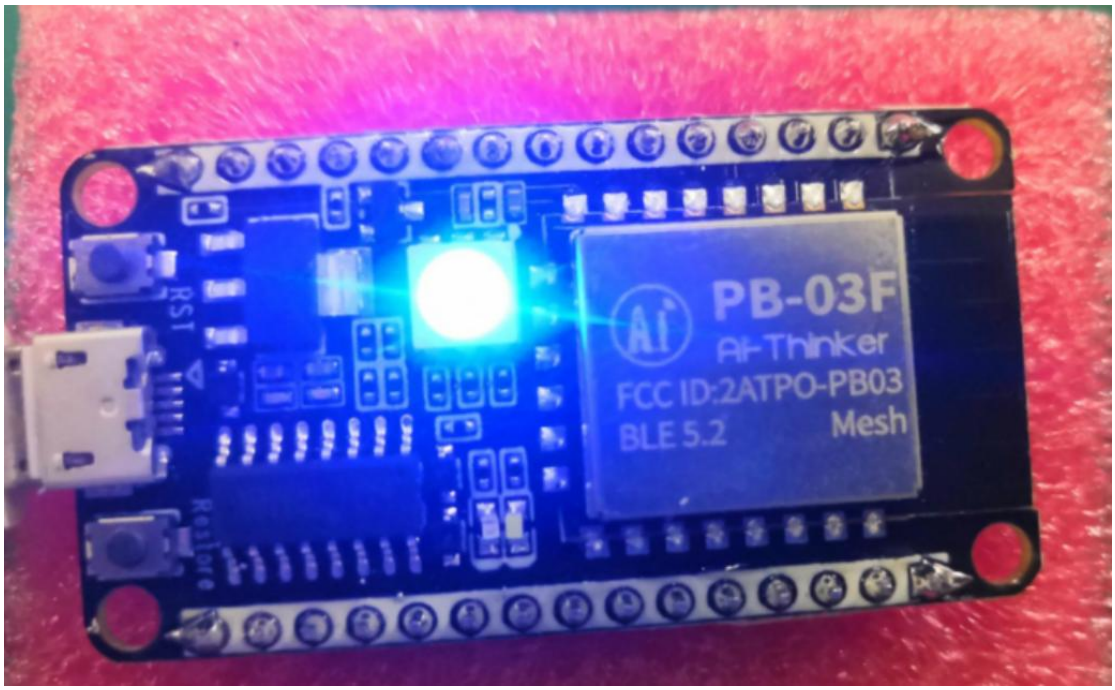
Flash success



After the download is successful, you need to manually restart the development board before the program will start execution.

5. Experimental phenomenon

The burning tool will output some debugging information.



Blue LED lighting.